Independent Branches for Verification Serving

Anonymous research draft

Abstract

Formal verification services check streams of program and proof candidates against formal specifications. Related candidates share libraries, declarations and accepted context. Independent attempts trade repeated preparation against shared execution, whose resource growth, contention and cancellation depend on candidate isolation. Native execution history includes scope, heuristics, runtime objects and resource counters beyond the logical context, so this tradeoff requires a separate argument about observable verification behavior. Established result caches, persistent environments, proof-state snapshots and bounded warm services recover substantial preparation reuse, while the value of independent native execution at equal resources remains unresolved. We argue that reusable verification state should retain preparation while separating each candidate's execution history and lifetime from preceding attempts. Realizing this separation requires supported quiescent runtime boundaries, faithful native configuration and resource handling, and bounded memory as candidates modify branch-local state. We present a verification service that retains immutable prepared states, executes disposable independent branches with inherited native resource state and separate service accounting, and uses native execution for unsupported boundaries. We compare complete verification in Lean and systems backed by satisfiability modulo theories (SMT) solvers on public candidate streams against competent bounded warm services and native logical or serialized prepared-state reuse, and assess remaining heterogeneous checking after preparation reuse on an RTX 5090 with full setup, transfer and CPU fallback costs. [**RESULT: End-to-end latency, throughput, memory and behavior results against the strongest matched services, including the measured scope of device benefit.**]

1 Introduction

Formal verification services check streams of program and proof candidates against formal specifications. Proof-generation systems submit related attempts that share libraries, declarations and accepted context. Services use proof assistants to construct and check proof terms, or verifiers to translate program obligations into requests to satisfiability modulo theories (SMT) solvers. Community tools for Lean [3] expose persistent environments for these workloads [6].

Serving independent attempts trades repeated preparation against shared execution. A separate verifier for each candidate repeats imports and parsing and replicates resident objects. Retained workers and shared logical environments avoid that repetition, but resource growth, contention and cancellation depend on worker isolation. Concurrent attempts against one imported Lean context illustrate the tradeoff: they need the same context and independent lifetimes. Cancelling one attempt must release its speculative work without ending the others.

This tradeoff depends on native execution history beyond visible formulas or declarations. Z3's combined solver can select a different solving path after an additional scope [25], and earlier requests can alter heuristics and resource counters. A proof assistant likewise retains elaborator state and mutable runtime extensions beyond its immutable declaration environment [3, 6]. Reconstructing a logically equivalent context therefore requires a separate argument for observable verification behavior.

Established reuse recovers preparation but leaves the value of independent native execution at matched resources unresolved. Result caches avoid repeating completed work [13], while the community read–eval–print loop (REPL) supports persistent environments, backtracking and pickling [6]. Recent Lean snapshotting retains elaborated proof state across parallel tactic branches [23], and stock Lean saves incremental elaboration snapshots between invocations [8]. Bounded warm services reuse environment roots and retire workers [20, 22]. Against these native-reuse baselines, the matched comparison must establish whether separating preparation from candidate execution history improves complete verification, cancellation and total memory across service processes.

We argue that reusable verification state should retain preparation while separating each candidate's execution history and lifetime. A context is prepared before candidate evaluation, and that native state remains available to independent attempts. Each candidate inherits the prepared state, so other candidates neither supply its solver history nor extend its lifetime. The value of this separation depends on context preparation cost, branch creation cost and the pages that each candidate modifies.

Realizing this separation requires runtime safety, faithful resource handling and bounded memory. A branch boundary must be quiescent and safe for the veri-

fier runtime, and native configuration and resource accounting must remain faithful when process creation changes timers or descriptors [15]. Preparation sharing must improve service capacity within a memory budget that bounds retained states. These requirements connect compatibility to the service's resource frontier, the achievable combinations of latency, throughput and memory use at matched CPU capacity and memory limits.

We present a verification service that retains immutable prepared states in a parent performing no candidate verification and executes disposable independent branches. Each branch owns its responses, mutable state, communication and cancellation. It inherits native resource state and receives separate service accounting. The service addresses runtime compatibility and memory use through supported quiescent boundaries, untouched candidate operations after preparation and bounded prepared-state lifetimes, and uses native execution for unsupported boundaries. We compare complete verification on public Lean and SMT-backed candidate streams with competent bounded warm services and native logical or serialized prepared-state reuse. After preparation reuse, we assess the remaining heterogeneous checking on an RTX 5090 with setup, transfer and CPU fallback included in complete verification costs. **[RESULT: Matched behavior results and end-to-end latency/throughput/memory findings, including the measured scope of device benefit.]**

This paper makes four contributions.

- A characterization of expensive preparation sharing in actual ordered verification attempts (Section 6.2).
- A native branching design and an argument about observable behavior separating preparation from candidate interference (Section 4).
- A bounded implementation integrating independent branches with native verification and cancellation (Section 5).
- An equal-resource evaluation against established reuse, with a causal assessment of heterogeneous checking after preparation reuse (Section 6).

2 Background

2.1 Verification stages and outcomes

Lean source verification includes parsing, elaboration, proof construction and kernel checking [3]. Imports prepare declaration environments before a candidate is elaborated, while kernel checking validates proof terms against the dependent type theory [3]. By contrast, post-export checker replay begins after the source frontend has already run. These boundaries distinguish savings in candidate construction, library setup and proof checking [9, 6].

SMT-backed verification includes frontend translation, solver setup, parsing, preprocessing and solving [5, 25]. A native SMT session can contain multiple queries and output requests. Solver responses include SAT for satisfiable queries, UNSAT for unsatisfiable queries, and UNKNOWN when the solver does not decide the query. Clients can also request models, proofs or unsatisfiable cores when the configured backend supports them [25]. These outcomes and artifacts define the comparison in Section 4.2.

An accepted Lean command may admit an unproved obligation using `sorry` [6]. Complete proof verification distinguishes unfinished proof obligations and admissions from frontend diagnostics.

Closing a tactic's goals is also distinct from checking the enclosing modified declaration. LeanPolish tests tactic proposals against saved proof states before re-elaborating selected source edits [1]. Its native menu screen restores state without separately finalizing each alternative as the original declaration [12]. A successful tactic screen therefore establishes a different outcome from acceptance of the enclosing declaration or source.

2.2 Prepared native state

A native execution prefix is an ordered sequence of verifier operations together with its configuration and runtime state. For SMT, this includes options, declarations, assertions and scopes in their observed order [25]. For Lean, it includes imports and accepted commands that establish the environment or proof state before the next attempt. Persistent tools provide ways to retain some of this state [6, 23].

Native interfaces retain verification state, while runtime and resource properties determine how that state can be reused. The solver's own command interface saves state across evaluations [25], and a resource limit can bound internal work independently of elapsed time [26]. Process branching shares existing memory through copy-on-write, but child timers, descriptors and runtime threads need separate handling [15]. Section 4 uses these native-state and process properties to define preparation boundaries.

3 Motivation

3.1 Preparation and candidate lifetime

A service must account for preparation reuse and candidate lifetimes. Preparation has value when attempts reuse expensive state, rather than merely share similar text. **[RESULT: Final characterization of preparation CPU cost, memory and candidate fan-out from real Lean and SMT-backed streams.]** A service must also account for attempts that are rejected, remain incomplete or exhaust their resources. Their life-

times consume capacity even when the client seeks only one accepted candidate.

Established persistent sessions make cold process execution an insufficient performance baseline. The community REPL already allows multiple commands to refer to an earlier environment [6], and native proof-state snapshots support parallel tactic exploration [23]. We ask whether sharing native preparation changes concurrency, cancellation and aggregate memory relative to bounded versions of these services.

3.2 Scope and history effects

Logical equality of visible assertions does not establish equality of the native state. The distinction between logical context and execution history is observable in resource-bounded verification. [**RESULT: Final native-session and prepared-branch comparisons, including scope-only and prior-history controls for status differences.**] Scope effects can select a different solver path even in a fresh process [25], while history effects can arise without a change to the candidate's explicit assertions. A compatibility argument must distinguish both causes from timing variability and account for native resource and runtime properties when establishing a faithful branch boundary.

The preparation/isolation tradeoff and scope/history distinction motivate the design in Section 4. Preparation sharing, candidate isolation and bounded resource use are evaluated together. A speedup from a path that changes the verifier responses or requested artifacts is excluded from the service comparison, even when that path remains logically sound on some inputs.

4 Design

The design has three goals derived from Section 3: (1) preserve the native verifier's observable behavior, (2) share expensive preparation while isolating candidate lifetimes, and (3) improve latency and throughput within matched CPU and memory resources. The service treats a prepared state as an immutable branching point. Candidate evaluation occurs only in an independent branch, and unsupported boundaries use native execution.

Placement first chooses whether retaining preparation is worthwhile, and requests without a retained state use native execution. For a selected prepared state, boundary qualification separately determines whether branching is supported (Figure 1). The parent owns the immutable preparation, while each supported branch owns candidate mutation and its lifetime. Supported branches execute on CPU with supported device operations where available, while unsupported boundaries use native execution.

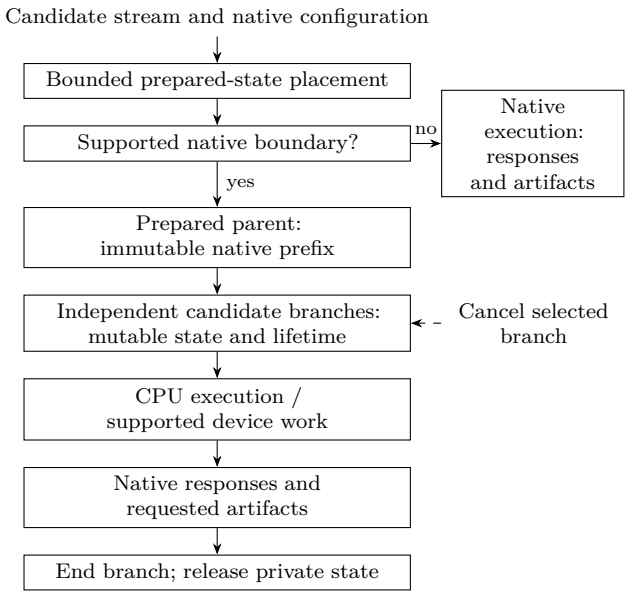


Figure 1: Supported branches inherit an immutable prepared prefix and own candidate mutation and lifetime. The parent performs no candidate verification after the selected boundary. Cancelling a selected branch ends its private work while leaving the parent and other branches independent. Normal completion releases private state after returning native responses. Unsupported boundaries use native execution. Placement can also select native execution without retaining a prepared state.

4.1 Preparation boundaries

A shared parent must represent state that every assigned candidate has available. The service therefore matches the original ordered preparation and configuration, without replacing them with a set of assertions. A candidate suffix begins after this preparation and includes its original scopes and output requests. The parent retains accepted context in the original preparation, but contains no learned or elaborated state produced by candidate suffixes after the selected boundary.

The boundary must also be safe for the runtime. Each adapter exposes a supported quiescent point with no pending candidate work or unresolved external effects. The service retains the prepared objects and initializes branch-owned communication and resource controls before evaluation. A runtime without such a point follows the native path. This restriction is a deployment boundary, not evidence that arbitrary native processes can be cloned safely.

4.2 Observable behavior

Native behavior is defined at the verifier interface, independently of the branch implementation. The reference is original native execution of the same ordered preparation and candidate suffix. This preparation retains the

candidate's accepted context without inserting suffixes from preceding alternative candidates. History within the candidate's own native session remains part of its suffix.

Behavior comparisons cover verifier outcomes and requested artifacts. For Lean, the comparison includes diagnostics, completed proof obligations and admissions in each attempt. For SMT, it includes each SAT, UNSAT or UNKNOWN response and each requested artifact. Native proof or model validation checks artifact meaning where alternative valid artifacts can differ in representation. Counting accepted commands alone does not determine fidelity between verification paths.

The candidate executes its original suffix once in a branch. The service inserts no additional logical scope and performs no preparatory solver check to warm heuristics. Native solver resource state is inherited from preparation, while service CPU and elapsed costs are charged separately. These rules remove cross-candidate history as a source of difference, but wall-time limits and changed runtime conditions still require measurement rather than a universal assertion that responses remain equal.

The service chooses fallback according to interface compatibility, rather than treating every UNKNOWN response as a detector of divergence. Returning UNSAT when the native resource-bounded path returns UNKNOWN can also change client behavior. The service selects native execution when the adapter cannot support the boundary or requested artifact. Any residual mismatch in a supported comparison invalidates that path's fidelity claim. The service retains the mismatching case with its complete input and output.

4.3 Branch lifetime and cancellation

Candidate mutation should not force the service to retain the shared parent beyond its configured policy or affect another candidate. Each branch owns mutable candidate state and terminates after producing its response, while cancellation stops that branch and releases its private memory. Device work and buffers remain attributable to their candidate until completion or cancellation cleanup, which is included in branch lifetime and service accounting. The prepared parent remains available only while it satisfies the configured retention policy.

The lifecycle separates retained preparation from speculative work. A cancelled or incomplete attempt cannot become a new prepared parent. Branch failure returns an explicit native or service failure and leaves other branches independent. The evaluation includes failed attempts and cancellation recovery rather than reporting only successful candidate throughput.

4.4 Bounded placement and execution

The benefit of preparation sharing must offset the costs of retained memory and branch mutation. The service admits prepared states within a memory budget and accounts for proportional shared memory and private dirty pages. Proportional set size (PSS) attributes each shared page across the processes sharing it [14], and fleet memory sums these attributed amounts across service processes. Placement favors states whose observed preparation work is reused often enough to offset the costs of creating and retaining those states. Branch concurrency is limited by CPU capacity and memory pressure, so copy-on-write sharing does not justify unlimited fanout.

If candidates modify most retained pages, copy-on-write sharing can approach full replication. If preparation is cheap or locality is low, native execution avoids retention overhead. These cases test the same service tradeoff as workloads with substantial preparation sharing and define its measured operating range.

4.5 Heterogeneous checking

When preparation reuse removes repeated work, it leaves a smaller verification path for an accelerator to improve. The service selects expensive supported operations from profiles of that path and compares them with the strongest CPU implementation. For these checking operations, the input expression graph alone does not define dependencies, because reduction depends on binder environments and instantiated constants [3, 11]. A GPU operation therefore includes its actual dynamic work, required representations and synchronization.

Supported device work preserves verifier semantics. Before a device operation is admitted, device qualification compares its outputs with matched CPU references. During execution, enclosing native verification checks the resulting proof or requested artifact where one is produced, and required online checks are charged to complete verification. Qualification does not imply a second full CPU replay on every request. Response and artifact fidelity follow Section 4.2, including UNKNOWN and responses without artifacts.

Unsupported work returns to CPU execution, whose cost remains in the complete verification measurement. Complete verification includes device initialization, representation construction, upload, download and queueing. We report operation speedups measured with data already on the device separately from complete checking.

5 Implementation

The system implements the design through native verifier adapters, process branches and a bounded service. The SMT adapter retains Z3's native command context

across native parser invocations to evaluate ordered session state [25]. The Lean adapter uses the runtime boundary that prepares imports and accepted proof context before a candidate. For each verifier, qualification first compares fresh adapter execution with the original interface and then compares prepared branches with that reference. We measure reuse after these comparisons.

5.1 Native adapters

The native SMT command adapter is implemented in C++. It retains the command-line interface's command context and preserves the native order of configuration, manager creation and solver creation while evaluating original command sequences. It preserves options, scopes, checks and requested artifacts rather than reconstructing assertions through a different solver API.

Qualification compares fresh adapter execution with the matched standalone executable, using an unmodified executable built from the same source to distinguish build metadata from other response differences. This comparison separates interface incompatibility from differences caused by branching. Every observed response difference remains visible. Diagnostic branching tests can investigate a rejected adapter, but the service executes prepared-state branches only with an adapter that passes interface qualification.

A preparation boundary can separate parser invocations while retaining the native command context. Qualification therefore compares complete input, split native parsing and branch evaluation to distinguish parser end-of-input effects from branch effects.

For source verification backed by this SMT adapter, the measured interval also includes the native frontend's translation and its original solver interaction. Solver-session replay alone is reported at its narrower boundary.

For Lean, the adapter retains accepted context and keeps branch-local proof construction independent. It waits for source snapshots and the checked declaration environment before qualifying runtime quiescence, thread ownership and mutable extensions. Frontend completion alone does not establish that runtime workers, their finalizers and event loops have stopped. The adapter enables native process branching only after that separate runtime qualification.

5.2 Process and resource handling

Linux copy-on-write branches retain the parent address space, with candidate writes creating private pages [15]. The parent stays quiescent and never executes candidate suffixes, while branch communication uses separate descriptors. The service reaps children after response, cancellation or failure and distinguishes native solver resource counters from service elapsed and CPU accounting.

Cancellation instrumentation records the native progress reached before termination and whether the child was reaped before later work. The instrumentation establishes reaping after native progress, while evidence of interruption inside a check requires separate execution-phase records.

Qualification of supported boundaries covers native resource controls affected by process creation. Claims about wall-time outcomes are limited to the measured operational scope described in Section 4.2 and the evaluation limitations, rather than following from inherited counters alone.

The service bounds prepared-state retention and branch concurrency using the preparation-reuse criterion in Section 4.4. Instrumentation records parent and child proportional memory, private dirty memory and process lifetimes. This accounting distinguishes shared preparation from full worker replication and includes replacement costs.

5.3 Device path

The device path uses an allocated RTX 5090 through NVIDIA's container runtime [18]. During device qualification, a CPU reference and GPU implementation consume the same identified input and preserve output identity for each supported checking operation. Required online checks follow Section 4.5. Native artifact fidelity follows Section 4.2, and complete verification includes CPU fallback and device/runtime costs. For checker representation, existing techniques include expression layouts with integer identifiers and delayed substitution [11, 10], so their use does not establish a new checking algorithm.

The Linux implementation integrates native SMT session evaluation in C++, Lean context reuse and matched CPU/device execution through these adapters.

6 Evaluation

The evaluation answers four research questions, each mapped to the design goals and contribution it tests. RQ1 tests the sharing opportunity characterized by the first contribution. RQ2 tests behavior preservation and isolation, the first two design goals, while RQ3 tests the third goal, the equal-resource service frontier. RQ4 tests whether heterogeneous execution improves that frontier for complete verification after reuse.

RQ1 How much expensive native preparation can real verification candidate streams share across independent jobs?

RQ2 Does branching from prepared native state preserve verifier responses and requested artifacts while isolating candidates and cancellation?

RQ3 How does prepared-state branching change end-to-end latency, throughput and memory at equal

resources compared with established warm-session reuse?

RQ4 After preparation reuse, when does heterogeneous execution improve complete verification over the strongest measured CPU path?

6.1 Experimental setup

We use complete ordered public proof and program attempts from multiple contexts in Lean and SMT-backed systems verification. Published proofs and deliberate invalid or incomplete variants provide behavior controls, while failed generated attempts remain part of the candidate-stream evaluation. These controls do not establish locality in generated candidate traffic. VeruSAGE provides real repository-level tasks [17, 24] for Verus [5], and Mathlib and the Lean Kernel Arena provide source/library and checker workloads [7, 9]. **[RESULT: Final source-native candidate-stream identities, versions, complete coverage and exclusions.]**

The main service competitors are bounded warm services with their existing cache and environment mechanisms, and native logical or serialized prepared-state reuse [20, 6, 8]. Existing REPL environment backtracking and native logical proof-state branching remain baseline implementations. Fresh native execution supplies the behavior reference and a preparation control. The heterogeneous comparison uses the fastest runnable complete CPU checker at a matched boundary, in addition to official Lean. A kernel checker operates after export only when the measurement explicitly uses that boundary. **[RESULT: Pinned strongest service artifacts and matched configuration.]** Baselines receive the same candidate information, CPU capacity, memory budget and admission order.

We preserve all raw per-input outcomes. Lean outcomes include unfinished proof obligations and admissions as well as frontend diagnostics. A successful tactic screen requires the enclosing declaration or source checking stage before it counts as an accepted proof.

Primary metrics are end-to-end candidate latency distributions, completed attempts per second, corpus makespan, CPU time and fleet proportional memory. Queueing, preparation, parsing, elaboration, proof construction, checking, transport, setup and teardown are included where they occur. Throughput cost per completed job is distinguished from request latency.

We repeat whole comparisons with alternating condition order. Affinity specifies permitted CPUs but alone does not provide CPU isolation [16], so we record affinity with topology and observations of competing work. **[RESULT: Hardware, topology, software versions, repetitions, observed load, uncertainty and measurement details.]**

6.2 Preparation sharing

RQ1 asks how much expensive native preparation real verification candidate streams share across independent jobs. We preserve original attempt ordering and measure preparation CPU time and resident state available before each attempt. We separately analyze result reuse for exact duplicate attempts, sharing of ordered native prefixes and sharing of native environments. Sharing a native environment characterizes established reuse, while a supported process branch must also satisfy the preparation and boundary conditions in Section 4.1. The characterization includes failed and incomplete attempts, so locality is measured across all attempts rather than only successful proofs.

We compare original ordering with a control containing unrelated work and measure how preparation depth changes reusable work. Text bytes and declaration counts are descriptive inputs, while measured preparation cost determines the opportunity. **[RESULT: Preparation-cost distributions, context reuse over time and aggregate achievable sharing with uncertainty.]** RQ1 remains unanswered until these measurements of complete original candidate streams establish whether preparation sharing is consequential.

6.3 Behavior and isolation

RQ2 asks whether branching from prepared native state preserves verifier responses and requested artifacts while isolating candidates and cancellation. We first compare fresh adapter execution with the original verifier, then execute untouched candidate suffixes from prepared states. We retain per-input decisions and artifacts for comparisons that vary preceding unrelated attempts, branch cancellation and resource-limit conditions. Completed attempts are compared for native responses and artifacts. Externally cancelled attempts are compared for progress at termination, cleanup and the independence of surviving attempts. Native scope and history controls distinguish changes in solver mode from cross-candidate interference.

Lean controls cover complete acceptance, rejection, unresolved obligations and admissions. SMT controls cover SAT with model requests, UNSAT with supported proof/core requests, UNKNOWN and multi-check sessions. The original scope/history regression streams remain part of the causal test. **[RESULT: Complete per-input comparisons, artifact validation, resource-limit outcomes and cancellation recovery.]** RQ2 remains unanswered until every planned behavior comparison finishes with a recorded outcome, residual differences are explained and divergent paths remain excluded from qualified service comparisons.

6.4 Latency, throughput and memory

RQ3 asks how prepared-state branching changes end-to-end latency, throughput and memory at equal resources compared with established warm-session reuse. We replay actual attempt streams over offered-load and concurrency sweeps, matching CPU and memory budgets across services. A single shared warm worker is not the sole comparison to many branch workers. We charge the measured service for creating and replacing prepared states and for cache costs.

We report latency distributions and completed throughput together with fleet proportional memory and CPU work. The causal comparison includes startup-only branches, deeper prepared prefixes and independent warm workers. Ablations remove prefix preparation, vary preparation depth and replace placement with ordinary bounded retention. Private dirty pages and branch lifetime explain whether sharing or simple warm startup drives the observed gain. [**RESULT: Matched frontier plots, tail latency, memory, cancellation cost and mechanism ablations.**] RQ3 remains unanswered until branching is compared with the strongest competent services over the complete workload and resource matrix.

6.5 Heterogeneous execution

RQ4 asks when heterogeneous execution improves complete verification over the strongest measured CPU path after preparation reuse. We identify whether each complete measurement begins with source verification or post-export checking. Results from post-export checking therefore apply only to that checking boundary. We profile warm complete checking and identify costly operations and their actual context-sensitive dependencies. Device qualification runs the same supported operation on CPU and RTX 5090 with exact reference comparisons, while required online checks follow Section 4.5. All unsupported work remains in the complete verification interval.

We separate host representation construction, device setup, transfers, checking with data already on the device, synchronization and fallback. The complete CPU/device comparison uses the service accounting and matched workload boundary of RQ3. An improvement in a checking operation alone therefore does not establish an improvement in the service frontier. The comparison includes tuned alternative CPU checkers with different parsing and reduction algorithms [11, 10]. [**RESULT: Cost coverage, amount of checking work simultaneously ready to execute, complete CPU/device comparisons and workload crossover.**]

Complete source-native ProofNet comparison. Three complete primary comparisons process the same 5,952 retained ProofNet-lineage [21, 4] Lean4 source attempts in original order, including failed attempts. Each method uses four frontend workers on the same four permitted CPUs, the retained Lean4.9.0-rc1/Mathlib environment, default native runtime threads and the original 300-second per-attempt allowance. The GPU method uses an RTX 5090 and includes conservative host admission, packet construction, device initialization, transfer, synchronization, CPU fallback, enclosing source verification and final native checking. Lossless experimental archival occurs after the measured final EOF. CPU affinity does not provide exclusive cores; GPU time slicing shares the device with another Workspace.

Whole corpus time (s)	Run 1	Run 2	Run 3
Native warm pool	310.478	311.544	318.572
Native snapshot/cache	319.524	320.059	331.712
Packed CPU control	319.510	326.852	321.050
GPU provider	315.870	322.891	323.607

Table 1: Complete source verification for 5,952 ProofNet-lineage attempts per condition. All twelve primary conditions finish with identical basic outcomes within each matched block. Four separate resource profiles also complete. The complete miniF2F cohort comparison is pending.

Table 1 rejects an end-to-end acceleration claim for this supported Meta reduction mechanism on the measured cohort. Relative to the faster native baseline in each block, GPU execution is 1.74%, 3.64% and 1.58% slower (mean 2.32%; observed range, not a confidence interval). Every condition has 413 complete, 5,534 rejected, four incomplete and one retained operational outcome. The packed CPU version is a component control: its negative-admission placement differs from the GPU path, so their time difference cannot isolate device placement alone. The strongest native baseline grounds the end-to-end comparison.

Native operation/readback logs establish actual GPU engagement, while unsupported work remains native. They do not establish full GPU kernel discharge. Missing terminal/interleaved records censor operation totals; nested provider timers are not exclusive fractions of whole corpus time. [**RESULT: Final repeated miniF2F comparison, supported-work coverage and complete workload-bound interpretation.**]

Four separately instrumented complete conditions observe whole times of 328.046 seconds for warm execution, 348.390 for snapshot/cache, 338.670 for the packed CPU control and 342.949 for GPU execution. Their sampled fleet PSS maxima are respectively 5,289,626, 9,268,751, 5,161,250 and 6,315,211 KiB. These are single instrumented conditions: one-second non-atomic sampling misses short-lived processes and peaks, and the sampling process contributes CPU overhead. These clocks do not replace the primary comparisons or establish replicated memory uncertainty; GPU context/driver

memory and other owners are outside host-PSS figures. The remaining workload and strongest-checker comparisons remain necessary to answer RQ4 at its stated scope. RQ4 remains unanswered until measured complete boundaries establish a device benefit or a negative opportunity bound.

6.6 Limitations

The service applies at supported native boundaries and retains the verifier as the authority. Supported backend versions and mutable runtime extensions constrain deployment. Logical equivalence alone does not promise equal resource-bounded responses. Wall-time limits depend on machine load and execution speed, so the evaluation states their operational scope separately from deterministic work limits. Valid negative and mixed results bound the claimed operating range.

Public generated attempts are replay workloads, whose arrivals can differ from a deployed multi-user service. Fixed published proof corpora remain useful controls but do not establish real candidate fan-out. Full GPU Lean checking is a distinct result from acceleration of a metadata primitive, and GPU certificate proving is a distinct workload from ordinary proof checking.

7 Discussion

Separating preparation from candidate execution connects native behavior to service resource use. Sharing preparation can reduce replication, while each candidate's mutation and lifetime determine how much of that sharing survives. The matched comparison therefore accounts for behavior, cancellation and fleet memory together, rather than treating retained context alone as the service benefit. Extending the service to another verifier requires a supported runtime boundary and qualification against its original interface.

8 Related work

Persistent verification state. Persistent Lean environments and native proof-state snapshotting retain elaborated context across attempts [6, 23]. Stock Lean also saves full elaboration or post-import snapshots [8], while Kimina provides bounded warm verification with environment reuse [20, 22]. These mechanisms provide the direct comparison for preparation reuse. The service evaluates independent native execution, cancellation and bounded fleet resources in addition to logical environment branching. This distinction requires matched evidence, rather than a novelty claim for snapshots themselves.

LeanPolish already restores native tactic state, resumes unchanged commands from saved checkpoints and supports persistent import workers [12]. Its released proof-improvement attempts include failed alternatives, while menu screening and later source verification have different boundaries [1]. These existing mechanisms strengthen the native-reuse comparison. Failure collection and saved-state execution are established baseline capabilities.

Verification caching. Fine-grained verification caching avoids rechecking unchanged program regions [13]. The service comparison combines competent caching with warm execution, so cold startup cannot account for the entire claimed advantage. Caching and session reuse retain different parts of verification work. For session reuse, native SMT interfaces support incremental command histories and requested artifacts [25]. Reconstruction with equivalent assertions remains a different execution history from the original native stream.

Prepared execution. Prepared execution and copy-on-write sharing are established systems mechanisms. SOCK and SEUSS share prepared state to reduce startup and increase parallel service density [19, 2]. A contribution beyond these prepared-execution systems requires more than specialization to verification. Our comparison tests whether native behavior and bounded shared preparation together require and benefit from a verification-specific execution mechanism.

Heterogeneous checking. GPU and CPU checker representations address different parts of verification. Nanoclo uses delayed substitutions, and nanoclo-fortran stores kernel objects in parallel arrays with integer identifiers [11, 10]. These checkers are direct competitors to representation-only acceleration. Our heterogeneous evaluation asks about the expensive dynamic checking that remains and complete verification costs, rather than extrapolating from input graph width.

9 Conclusion

Native preparation and candidate execution history are separate resources in verification serving. Independent branches retain preparation while isolating candidate lifetimes, and their value depends on behavior fidelity and the equal-resource service frontier. **[RESULT: Final findings answering all four RQs, including the measured heterogeneous scope.]** The matched comparison determines whether branching supplies a systems benefit beyond existing warm reuse.

References

[1] Pauline Bourigault. Leanpolish: Verified supervision for lean proof compression, 2026. arXiv preprint, version 1.

[2] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. Seuss: Skip redundant paths to make serverless fast. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys)*, 2020.

[3] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[4] Goedel-Prover contributors. Goedel-prover proofnet dataset, 2026. Source revision d80349d93a80305765492014f0b395cec46e5023; accessed 2026-10-14; year denotes access year.

[5] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), 2023.

[6] Lean community. Lean community repl, 2026. Accessed 2026-10-07; year denotes access year.

[7] Lean community. Mathlib, 2026. Accessed 2026-10-07; year denotes access year.

[8] Lean contributors. Lean 4.32.0 release: incremental snapshots, 2026. Accessed 2026-10-07; year denotes access year.

[9] Lean contributors. Lean kernel arena, 2026. Accessed 2026-10-07; year denotes access year.

[10] Lean Kernel Arena. Nanoclo-fortran: Lean kernel arena checker, 2026. Accessed 2026-10-07; year denotes access year.

[11] Lean Kernel Arena. Nanoclo: Lean kernel arena checker, 2026. Accessed 2026-10-07; year denotes access year. Checker modifications written by Claude, directed by Sebastian Graf; upstream components credited on the page.

[12] LeanPolish contributors. Leanpolish source implementation, 2026. Revision 70cda81a15cca1f805885df197361102f0123cbb; accessed 2026-10-09.

[13] K. Rustan M. Leino and Valentin Wüstholz. Fine-grained caching of verification results. In *Computer Aided Verification (CAV), Part I*, pages 380–397, 2015.

[14] Linux kernel contributors. The /proc filesystem, 2026. Accessed 2026-10-07.

[15] Linux man-pages project. fork(2) linux manual page, 2026. Accessed 2026-10-07; year denotes access year.

[16] Linux man-pages project. sched_setaffinity(2) linux manual page, 2026. Accessed 2026-10-09.

[17] Microsoft Research. Verusage-bench, 2026. Accessed 2026-10-07; year denotes access year.

[18] NVIDIA contributors. Nvidia container toolkit, 2026. Source revision a672e378ef91bea7dfe92c0ed274847fa1cb9236; accessed 2026-10-09.

[19] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Sock: Rapid task provisioning with serverless-optimized containers. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 57–70. USENIX Association, 2018.

[20] Project Numina. Kimina lean server, 2026. Accessed 2026-10-07; year denotes access year.

[21] ProofNet contributors. Proofnet benchmark source, 2026. Accessed 2026-10-14; year denotes access year.

[22] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[23] Austin Shen and Yunong Shi. Keep the proof state live: Snapshotting for efficient tactic search in lean 4, 2026.

[24] Chenyuan Yang, Natalie Neamtu, Chris Hawblitzel, Jacob R. Lorch, and Shan Lu. Verusage: A study of agent-based verification for rust systems, 2025.

[25] Z3 contributors. Z3 c api, 2026. Accessed 2026-10-07; year denotes access year.

[26] Z3 contributors. Z3 guide: Parameters, 2026. Accessed 2026-10-07; year denotes access year.